Layer10 Take-Home Project Write-up
Memory Graph from GitHub Issues (Apache Airflow)

Darshan Sonawane

March 8, 2026

Abstract

This project implements a grounded long-term memory system that transforms unstructured project discussions into a structured knowledge graph. Using Apache Airflow GitHub Issues as the corpus, the system extracts entities, claims, and supporting evidence from issue metadata and comments through a hybrid pipeline combining deterministic rules and LLM-assisted extraction.

The extracted knowledge is deduplicated, canonicalized, and stored in a Neo4j memory graph where claims are linked to evidence excerpts from the source artifacts. A retrieval component maps natural-language queries to candidate entities and claims, ranks them using confidence and evidence signals, and returns grounded context packs with citation-backed evidence.

An interactive Cytoscape.js visualization enables exploration of entities, claims, and supporting sources. The design emphasizes correctness over time through temporal validity tracking, evidence grounding, and reversible deduplication, while remaining extensible to organizational environments such as email, Slack, and Jira.

Contents

1 Project Overview **2**

2 Ontology / Schema **2**

3 Structured Extraction Pipeline **4**

4 Deduplication & Canonicalization **6**

5 Memory Graph Design & Storage **7**

6 Retrieval and Grounding **8**

7 Visualization **11**

8 Layer10 Considerations **12**

8.1 Ontology and Unstructured + Structured Fusion . . . 12
8.2 Extraction Contract Changes . . . 13
8.3 Deduplication Strategy . . . 13
8.4 Long-Term Memory Behavior . . . 13
8.5 Grounding and Safety . . . 13
8.6 Permissions . . . 13
8.7 Operational Reality . . . 13

9 Conclusion **14**

1 Project Overview

Corpus chosen. The project uses Apache Airflow GitHub Issues. This corpus was chosen because it has rich, structured metadata (author/state/labels/timestamps), unstructured text (issue bodies/comments), and linkable source URLs, which makes it well-suited for grounded memory-graph construction.

Exact source and reproduction. Data is pulled from GitHub REST API endpoint `/repos/apache/airflow/issues`. To reproduce the download and cleaning pipeline run the following scripts from the repository:

```
python3 Corpus/data_extraction.py
python3 Corpus/data_cleaning.py
```

The full project implementation, including extraction, deduplication, graph construction, retrieval scripts, and visualization layer, is available at:

https://github.com/luffy-taro-106/Layer10_Project

System summary. The system implements an end-to-end memory graph pipeline: ingest GitHub issues and comments, clean and normalize artifacts, extract deterministic and LLM-assisted claims, deduplicate claims, build canonical entities, ingest them into Neo4j as an Entity–Claim–Evidence graph, retrieve grounded context packs for questions, and visualize graph interactions using Cytoscape.js.

2 Ontology / Schema

The ontology defines the structured representation used to convert unstructured GitHub issue discussions into a grounded memory graph. The schema is designed to be deterministic, extensible, and compatible with graph-based storage.

Core model. The system represents memory as a graph composed of three primary objects:

- **Entities** — canonical objects such as issues, users, comments, labels, and states.
- **Claims** — typed relationships between entities representing extracted facts.
- **Evidence** — grounded source excerpts that support each claim.

Every claim must reference at least one evidence object derived directly from the original corpus.

Entity Types

Implemented entity namespaces/types include: Issue, Person, Label, Status, and Comment. PullRequest is supported in the type inference layer but is not currently populated in the output dataset. Entity JSON follows the schema defined in `data/Entities/schema_v1.json`.

.

Entity Type	Namespace	Example
Issue	`issue_{number}`	`issue_62799`
Person	`person_{login}`	`person_prajwal7842`
Comment	`comment_{id}`	`comment_3876779079`
Label	`label_kind_{name}`	`label_kind_bug`
Status	`status_{state}`	`status_open`

Predicate Types

Claims represent relationships between entities. The current extraction pipeline produces the following predicate types:

- `ISSUE_CREATED_BY`
- `ISSUE_HAS_LABEL`
- `ISSUE_HAS_STATUS`
- `ISSUE_HAS_COMMENT`
- `COMMENT_WRITTEN_BY`
- `ISSUE_ASSIGNED_TO`

These predicates capture the most common structural relationships present in GitHub issue discussions.

Claim Schema

Claim JSON follows the schema defined in `data/claims/schema_v1.json`. A claim represents a structured assertion extracted from the corpus.

Required fields: `claim_id`, `subject`, `predicate`, `object`, `event_time`, `valid_from`, `confidence`, `extraction_metadata`, `evidence`.

.

Field	Type	Notes
`claim_id`	string	Deterministic UUIDv5
`subject`	string	Entity ID namespace
`predicate`	string	Ontology predicate
`object`	string/object/number	Usually namespaced string
`event_time`	string/null	Timestamp when event occurred
`valid_from`	string/null	Validity start of the claim
`valid_to`	string/null	Validity end (for historical updates)
`confidence`	number [0,1]	Confidence from extraction pipeline
`extraction_metadata`	object	Schema version, extractor version, model, prompt, extracted timestamp
`evidence`	object	Grounding payload describing supporting source

Evidence Schema

Evidence provides grounding for every extracted claim. Evidence payload fields include:

- `source_type`
- `source_id`
- `excerpt`
- `timestamp`
- `location`
- `offsets{char_start, char_end}`
- extraction metadata

During graph ingestion, each evidence object is converted into a dedicated `:Evidence` node, and the original source URL is reconstructed and attached as a property.

Normalization Rules

To ensure consistent entity resolution and prevent duplication, the system enforces deterministic namespace conventions implemented in `Extraction/claims.py` and the LLM extractor alignment logic.

```
person_{login}
issue_{number}
comment_{id}
label_kind_{name}
status_{state}
```

All identifiers are normalized to lowercase and sanitized to maintain deterministic graph keys.

Temporal Semantics

Two temporal fields are maintained for each claim:

- **event_time** — when the event occurred in the source system.
- **valid_from / valid_to** — the time interval during which the claim is considered true.

This allows the system to distinguish between current and historical facts and represent changes such as issue state transitions or reassignment.

Example Claim

```
{
  "claim_id": "3fbfd60e-7467-5581-b01b-504da3b3c008",
  "subject": "issue_62799",
  "predicate": "ISSUE_CREATED_BY",
  "object": "person_prajwal7842",
  "event_time": "2026-03-03T16:35:33Z",
  "valid_from": "2026-03-03T16:35:33Z",
  "valid_to": null,
  "confidence": 1.0,
  "extraction_metadata": {
      "schema_version":"1.0",
      "extractor_version":"deterministic_v1"
  },
  "evidence": {
      "source_type":"issue_metadata",
      "source_id":62799,
      "excerpt":"..."
  }
}
```

The schema is designed to remain extensible so additional entity types (e.g., PullRequest, Repository, Project, Component) and new predicates can be incorporated without altering existing graph structure.

3 Structured Extraction Pipeline

The structured extraction pipeline converts raw GitHub issue artifacts into typed, grounded claim objects stored in the memory graph. The design prioritizes deterministic extraction, strict schema contracts, and evidence grounding so that extracted knowledge remains auditable and stable over time.

Pipeline flow. Issue JSON retrieved from GitHub is cleaned and normalized before being processed by two complementary extraction stages: deterministic rule-based extraction and LLM-assisted extraction for assignment relationships. Extracted claims are then validated, deduplicated, and converted into canonical entities before graph ingestion.

```
GitHub -> raw JSON -> preprocessing -> deterministic + LLM claims
-> schema validation -> claim dedup -> entity build
```

Extraction Contract

Each extracted claim must satisfy a strict schema contract containing:

- subject entity identifier
- predicate type
- object entity identifier
- event timestamp
- confidence score
- extraction metadata
- at least one evidence payload

Extraction Methods

- **Deterministic extraction.** Issue metadata and comment structures are mapped to ontology predicates using rule-based transformations.
- **LLM extraction.** Used only for detecting assignment relationships in comment text. The prompt enforces a strict JSON output contract.
- The LLM predicate is restricted to `ISSUE_ASSIGNED_TO` and usernames must match explicit `@username` mentions.

Deterministic ID Strategy

Each claim is assigned a deterministic UUIDv5 generated from:

```
subject | predicate | object | event_time | source_id
```

This guarantees stable identifiers across repeated extraction runs, enabling idempotent updates and safe reprocessing.

Grounding

Every claim must reference supporting evidence from the original corpus. The evidence object includes:

- `source_type`
- `source_id`
- `excerpt`
- `timestamp`
- `location`
- `offsets {char_start, char_end}`

This grounding allows every stored claim to be traced back to its original source text.

Validation and Repair

Extraction outputs pass through validation before entering the memory graph.

- Schema validation using `jsonschema`.
- Deterministic normalization of entity namespaces.
- LLM outputs restricted to allowed predicates and valid JSON.
- Invalid JSON repaired by extracting embedded JSON fragments.

- Failed LLM calls retried up to `MAX_RETRIES = 2`.
- Claims missing required fields or evidence are discarded.

Versioning

Extraction metadata tracks the environment used to generate each claim:

- `schema_version`
- `extractor_version`
- `model_name`
- `prompt_version`
- `extracted_at`

This enables safe reprocessing and backfilling when the ontology or extraction logic evolves.

Quality Gates

To prevent noisy extractions from becoming durable memory, several quality controls are applied:

- confidence thresholds for LLM outputs
- predicate restrictions for LLM extraction
- claim deduplication with aggregated evidence

Handling Corpus Evolution

The extraction pipeline is designed to support incremental updates as the corpus changes. Because claim identifiers are deterministic and extraction metadata is versioned, the system can safely:

- reprocess updated issues
- backfill claims when the ontology evolves
- maintain historical claim validity intervals

This ensures that extracted knowledge remains consistent even as the underlying corpus changes.

4 Deduplication & Canonicalization

To prevent storing the same information multiple times, deduplication is applied at the artifact, entity, and claim levels. All merges record provenance so that decisions remain auditable and reversible.

Artifact Deduplication

During preprocessing, normalized issue and comment text is hashed using SHA256. If two artifacts produce the same hash they are treated as duplicates and linked through the field `is_duplicate_of`. This prevents duplicate messages from generating redundant claims while preserving the original source reference.

Entity Canonicalization

Entities are normalized using type-specific rules to collapse aliases and formatting differences.

- Person identifiers use normalized GitHub login names.
- Label names are lowercased and standardized.

When aliases resolve to the same entity, they are merged and recorded in `data/Entities/merge_log.json` with source, target, reason, and timestamp.

Claim Deduplication

Claims describing the same fact are merged using deterministic signatures:

- Non-temporal predicates: `(subject, predicate, object)`
- Temporal predicates: `(subject, predicate, object, valid_from)`

Claims in the same bucket are merged while aggregating their supporting evidence and increasing the `support_count`. Confidence values are combined using the rule:

$$1 - (1 - a)(1 - b)$$

Conflicts and Revisions

Temporal fields `valid_from` and `valid_to` represent historical changes such as status updates or reassignment. Earlier claims remain valid for their original interval, while newer claims represent the updated state. Conflicts are surfaced during retrieval when multiple objects exist for the same `(subject,predicate)`.

Reversibility and Audit

All merges record provenance so they can be inspected or undone.

- Entity merges are stored in a merge log with source and target entities.
- Claim merges preserve all supporting evidence rather than discarding duplicates.
- Support counts track how many artifacts confirm a claim.

Example Merged Claim

```
{
  "claim_id": "...",
  "subject": "issue_...",
  "predicate": "ISSUE_ASSIGNED_TO",
  "object": "person_...",
  "support_count": 3,
  "supporting_evidence": [ {...}, {...}, {...} ]
}
```

5 Memory Graph Design & Storage

Storage choice. Neo4j was selected because its property-graph model naturally represents entity–claim–evidence relationships and supports expressive querying through Cypher.

Core Graph Objects

The memory graph represents knowledge using four primary node types:

- **Entity** — canonical objects such as issues, users, labels, comment, and statuses.
- **Claim** — typed relationships representing extracted facts.
- **Evidence** — source excerpts grounding each claim.

Claims explicitly connect entities and are grounded by evidence.

- `(Entity)-[:SUBJECT_OF]->(Claim)`
- `(Claim)-[:OBJECT_OF]->(Entity)`
- `(Claim)-[:SUPPORTED_BY]->(Evidence)`

Node Properties

- Claim: `claim_id, predicate, event_time, valid_from, valid_to, confidence`
- Evidence: `evidence_id, source_type, source_id, excerpt, timestamp, url, offsets`

Time Representation

Two temporal fields capture both occurrence and validity:

- `event_time` — when the event occurred in the source system
- `valid_from / valid_to` — time interval during which the claim is considered true

Current facts are identified as claims with `valid_to IS NULL`.

Updates and Idempotency

The ingestion pipeline supports incremental updates. Deterministic identifiers combined with Neo4j `MERGE` operations ensure idempotent writes, allowing the pipeline to safely reprocess the corpus without generating duplicate nodes.

Handling Edits and Deletions

The current pipeline processes a static snapshot of the dataset. In a production system, updates such as edits or deletions would be handled as follows:

- **Evidence redaction:** If source text is deleted or redacted, the corresponding `:Evidence` node is marked as redacted instead of being removed.
- **Soft deletes:** Artifacts or entities are flagged with a soft-delete attribute.
- **Claim closure:** When a fact becomes invalid (e.g., reassignment or status change), the existing claim is closed by setting `valid_to` to the update timestamp.

Permissions (Conceptual)

Retrieval can enforce access control by filtering evidence nodes based on source permissions. Only claims supported by evidence accessible to the requesting user would be returned.

Observability

Pipeline execution logs ingestion counts and errors. Production monitoring would track metrics such as extraction success rate, invalid-output drop rate, deduplication reduction, ingestion latency, and query latency.

6 Retrieval and Grounding

The retrieval component answers questions by querying the memory graph and returning a grounded context pack containing relevant entities, claims, and supporting evidence.

Interface

Retrieval is implemented as a CLI script (`Retrieval/retrieval_grounding.py`) with an optional Streamlit interface. The module accepts a natural-language question and returns a structured context pack.

Mapping Questions to Candidates

Two candidate-generation modes are implemented:

- **Keyword mode**: regex extraction of issue IDs and usernames combined with identifier matching.
- **Hybrid mode**: keyword candidate generation followed by embedding-based reranking using Ollama embeddings.

This maps user questions to relevant entities and claims in the graph.

Ranking

Candidate claims are ranked using a scoring function combining confidence, recency, evidence support, and entity relevance.

```
score = confidence
      + 0.1 * min(evidence_count, max_evidence_per_claim)
      + recency_bonus
      + predicate_bonus
      + entity_bonus
```

Expansion and Pruning

Retrieval expands from candidate entities to connected claims and evidence while applying hard limits to avoid graph explosion.

- `max_claims_per_entity`
- `max_evidence_per_claim`
- `max_claims_total`

Predicate-aware selection maintains diversity while keeping the context pack compact.

Grounding and Citations

Every returned claim must be supported by evidence. Evidence excerpts are returned with citation identifiers (`[E1], [E2], ...`) linking to the original source metadata:

- source type
- source identifier
- excerpt text
- source URL

This ensures that all answers remain traceable to the underlying corpus.

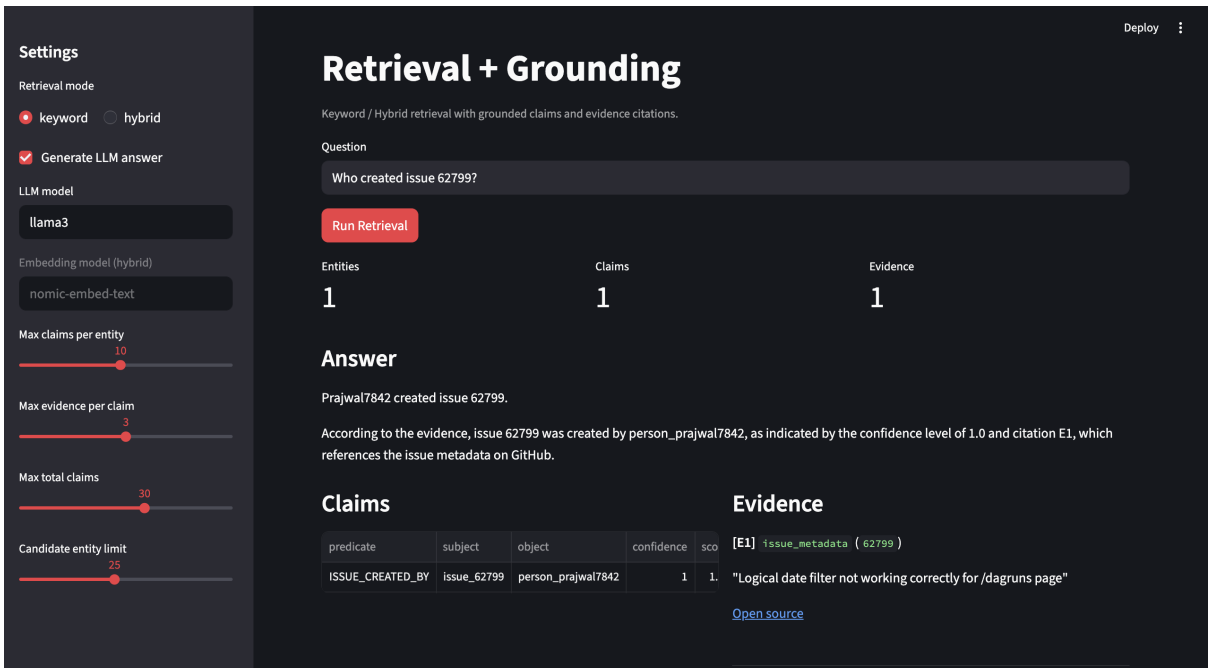
Ambiguity and Conflicts

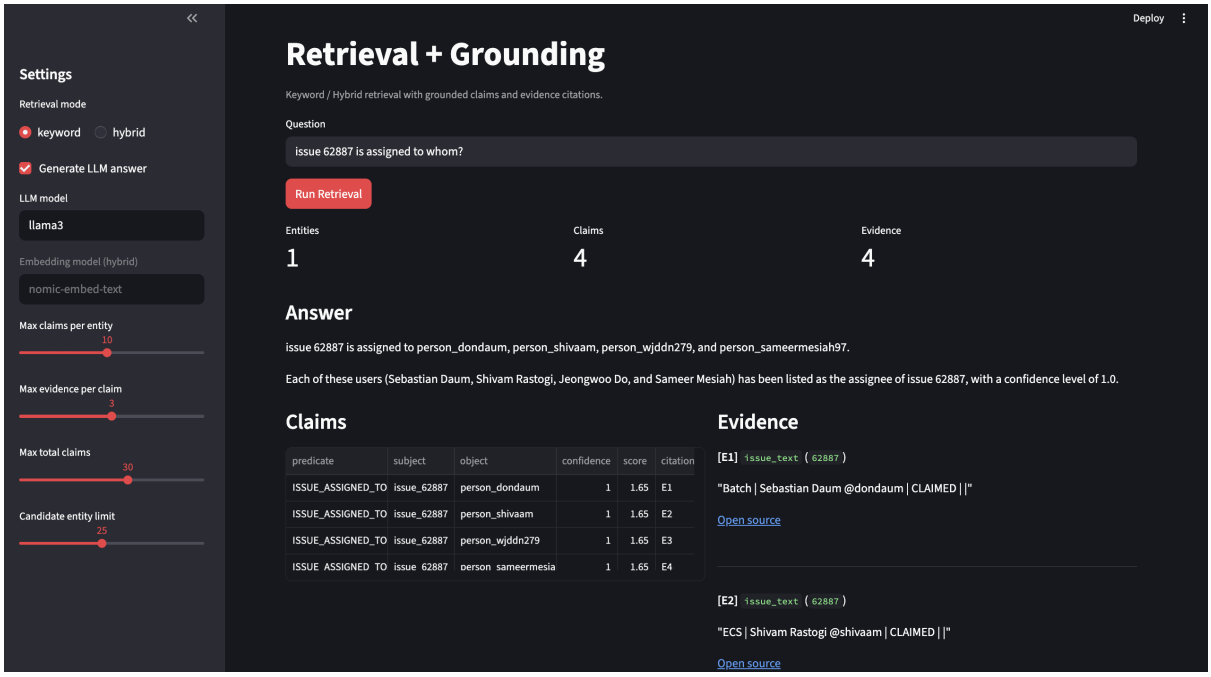
If a question is ambiguous (e.g., missing issue identifiers), retrieval returns multiple candidate entities. When multiple objects exist for the same `(subject,predicate)`, conflicting claims are returned together so downstream components or the user can resolve them.

Example Context Pack


```
{
  "question": "Who created issue 62799?",
  "entities": [{"id":"issue_62799","entity_type":"Issue"}],
  "claims": [{
    "claim_id": "...",
    "subject": "issue_62799",
    "predicate": "ISSUE_CREATED_BY",
    "object": "person_prajwal7842",
    "confidence": 1.0,
    "citations": ["E1"]
  }],
  "evidence": [{
    "citation_id": "E1",
    "source_type": "issue_metadata",
    "source_id": 62799,
    "excerpt": "Logical date filter not working correctly for /dagruns page",
    "url": "https://github.com/apache/airflow/issues/62799"
  }]
}
```


Example Queries and UI for Retrieval





7 Visualization

An interactive graph visualization is provided to explore the memory graph and inspect grounded claims. The visualization is generated by exporting the Neo4j graph into a Cytoscape.js interface using the script `Visualization/export_cytoscape_graph.py`, which produces a static interactive page (`cytoscape_graph.html`).

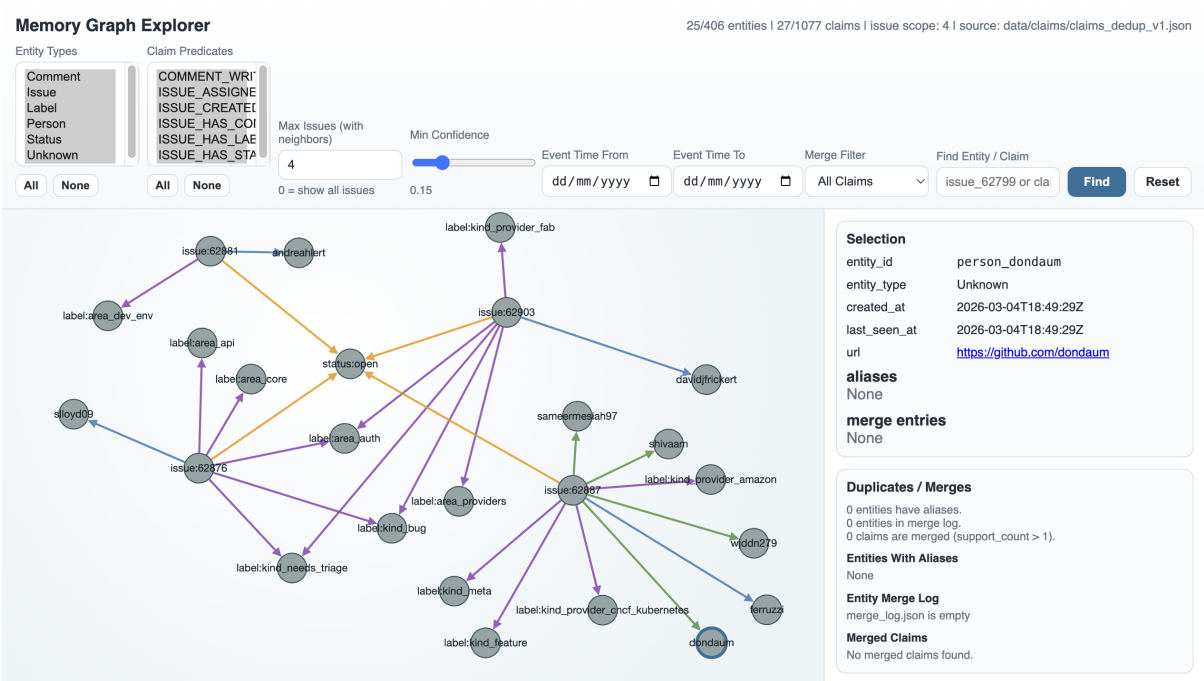


Figure 1: Interactive memory graph visualization showing entities, claims, and relationships

Figure 2: Claim inspection view displaying grounded evidence excerpts and source metadata

Key features

- Interactive graph exploration
- Entity type, Predicate-based, Confidence threshold, Event time range filtering
- Scoped graph view (max issues with neighbors)
- Entity / claim search
- Evidence inspection panel
- Source URL linking
- Duplicate / merge inspection
- Color-coded predicates
- Grounded evidence display

8 Layer10 Considerations

To adapt this system from GitHub issues to Layer10's environment (email, Slack/Teams, docs, Jira/Linear), I would keep the same core model (Entity–Claim–Evidence) and change the source adapters, ontology extensions, and retrieval controls.

8.1 Ontology and Unstructured + Structured Fusion

The ontology would be extended with source-native entities: `email_thread`, `message`, `doc`, `ticket`, `project`, `component`, and `team`.

Unstructured discussions (email/chat/docs) would be fused with structured systems by extracting and normalizing ticket IDs, URLs, project names, and component tags, then linking them to canonical Jira/Linear entities. This turns conversation artifacts into graph claims tied to operational objects.

Example. A Slack message saying *"This is fixed in AIR-1234, assigning to @alex for rollout"* becomes: `message_m987 -> ticket_AIR_1234`, `ticket_AIR_1234 -[ISSUE_ASSIGNED_TO]-> person_alex`, with evidence pointing to the original Slack permalink.

8.2 Extraction Contract Changes

The extraction contract remains strict but source-aware. Each claim should include: `subject, predicate, object, event_time, confidence, extraction_metadata, evidence`.

Evidence must include source-specific provenance fields (workspace/channel/thread/message ID or doc section anchor) so every claim is traceable to the exact source span.

Example. For email, evidence would include fields like `source_type=email`, `source_id=<message-id>`, `thread_id`, and excerpt offsets from the quoted email body.

8.3 Deduplication Strategy

Dedup would move from simple text hash to multi-signal dedup:

- message identity (platform IDs)
- near-duplicate text fingerprinting
- quote/reply stripping
- alias resolution for users/components/tickets

Claim dedup still uses semantic keys and aggregates supporting evidence.

Example. If the same Jira status update appears in Slack and email, both artifacts map to one claim: `ticket_AIR_1234 -[ISSUE_HAS_STATUS]-> status_done`, with two evidence entries (Slack + email).

8.4 Long-Term Memory Behavior

Durable memory should include stable facts: decisions, ownership, assignments, status transitions, and policy-level conclusions. Transient brainstorming and informal chatter stay ephemeral unless converted into grounded claims.

Drift is controlled by temporal validity (`valid_from`, `valid_to`) and by closing outdated claims when newer grounded evidence supersedes them.

Example. If a ticket owner changes from `person_alex` to `person_sam`, the old claim gets `valid_to` set, and a new owner claim starts with a new `valid_from`.

8.5 Grounding and Safety

All returned claims must remain citation-backed (excerpt + timestamp + source metadata + URL/deep-link). If content is deleted or redacted, associated evidence nodes should be marked redacted (or removed) while preserving an auditable claim trail and reason code.

Example. If a Slack message is deleted, the evidence node is marked `is_redacted=true, redaction_reason="source_deleted"`, while the claim remains for audit with degraded confidence or hidden from normal retrieval.

8.6 Permissions

Retrieval must be evidence-permission aware: a user can only see claims whose supporting evidence is accessible under their source ACLs (email mailbox, Slack channel, doc permissions, Jira project role).

Example. A user without access to `incident-private` should not receive claims grounded only in that channel, even if related Jira tickets are visible elsewhere.

8.7 Operational Reality

Productionization requires incremental ingestion, cost controls, and regression monitoring:

- incremental updates (delta pulls from each source)
- cost-aware model routing (deterministic first, LLM only where needed)

- evaluation gates (extraction quality, grounding coverage, dedup rate, retrieval accuracy)
- regression tests for schema, claim stability, and permission filtering

Example. Run hourly delta sync for Slack/Jira, then nightly regression checks to detect drops in grounding coverage or unexpected growth in conflicting claims.

9 Conclusion

This project demonstrates a grounded long-term memory system built from GitHub issue discussions. The system extracts structured claims from unstructured artifacts, deduplicates and canonicalizes entities, and stores the resulting knowledge in a queryable memory graph with evidence-backed claims. Retrieval returns grounded context packs for natural-language questions, and an interactive visualization enables exploration of entities, claims, and supporting evidence.

Future improvements include expanding the ontology to additional artifact types, improving extraction coverage for conversational sources, and introducing automated evaluation to monitor extraction and retrieval quality as the corpus evolves.